COMP 3920 Database Systems 1

MySQL
Creating Triggers

Introduction:
The goal of this document is to serve as a walkthrough creating triggers in MySQL 8.

We are going to be using a sample database for a Classic Model Store.
For more information: https://github.com/hhorak/mysql-sample-db

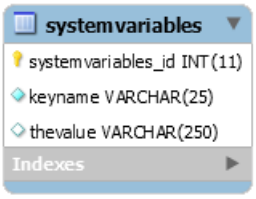
We will be creating a trigger that will keep track of the number employees in the employees table.
Also we'll create a trigger that will validate the input to a table to enforce data integrity.

Triggers can be added to any table and will fire (run) when that event happens. There are 3 different event types:

- INSERT
- UPDATE
- DELETE

We can run our code either BEFORE or AFTER the event has occurred. If we run our code BEFORE, we have the opportunity to check to see if the values being changed are correct and abort if the data would violate our database constraints. If we run our code AFTER, we can read all the values that get dynamically generated during the operation, i.e. some fields are calculated when a row is created or modified.

Step 1:
If you haven't already done so, create a `systemvariables` table in the `classicmodels` database as such:



Insert into your new table the following row with the following query:

```
INSERT INTO systemvariables
(keyname, thevalue)
VALUES
('employeeCount', (SELECT count(*) FROM employees));
```

```
insert into systemvariables
(keyname, thevalue)
values
('employeeCount', (select count(*) from employees));

select * from systemvariables;
```

Result Grid | Filter Rows: | Edit: | Export/In

systemvariables_id	keyname	thevalue
1	nextProductId	5003
2	employeeCount	23
NULL	NULL	NULL

We've just created a system variable with the initial value containing the number of employees in the employees table.

Create the AFTER INSERT trigger:

```
DELIMITER $$
CREATE TRIGGER after_employee_insert
    AFTER INSERT ON employees
    FOR EACH ROW
BEGIN
    UPDATE systemvariables
    SET thevalue = thevalue + 1
    WHERE keyname = 'employeeCount';
END$$
DELIMITER ;
```

Create the AFTER DELETE trigger:

```
DELIMITER $$
CREATE TRIGGER after_employee_delete
    AFTER DELETE ON employees
    FOR EACH ROW
BEGIN
    UPDATE systemvariables
    SET thevalue = thevalue - 1
    WHERE keyname = 'employeeCount';
END$$
DELIMITER ;
```

Now let's test these triggers out:
Insert 2 rows into the employees table:

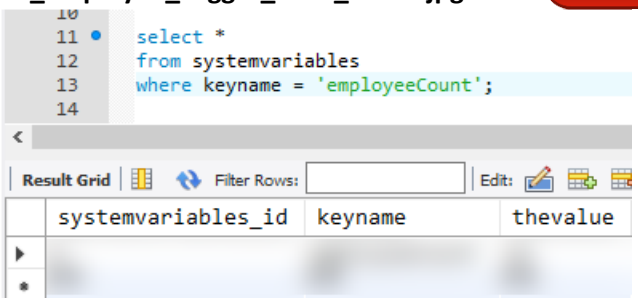
1705	Mary	Kim	x2323	mkim@classicmodelcars.com	4	1102	Sales Rep
1706	Zaf	Bal	x2324	zbal@classicmodelcars.com	5	1621	Sales Rep

Then execute the following query:

```
SELECT *
FROM systemvariables
WHERE keyname = 'employeeCount';
```

Submit your screenshot:
07_employee_trigger_after_insert.jpg

Submit



Delete the 1 of the rows you just added – Mary Kim (employeeNumber 1705):

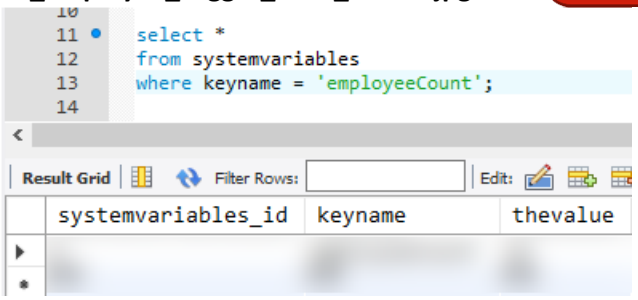
```
DELETE FROM employees
WHERE employeeNumber = 1705;
```

Then execute the following query:

```
SELECT *
FROM systemvariables
WHERE keyname = 'employeeCount';
```

Submit your screenshot:
08_employee_trigger_after_delete.jpg

Submit



Step 2:
For the products table we want to ensure proper data integrity.

We want to ensure the following:

- buyPrice > 0
- MSRP >= buyPrice

We will do this by combining a stored procedure and a few triggers.

Let's start by creating a stored procedure to enforce valid values.

```
DELIMITER $$
CREATE PROCEDURE `check_products`(IN buyPrice DECIMAL(10,2), IN MSRP
DECIMAL(10,2))
BEGIN
    DECLARE _messageText varchar(250);
    IF buyPrice < 0 THEN
           BEGIN
           SELECT CONCAT('Check constraint on products.buyPrice failed
- buyPrice ',buyPrice,' must be positive.')
           INTO _messageText;
        SIGNAL SQLSTATE '45001'
            SET MESSAGE_TEXT = _messageText;
           END;
    END IF;

    IF MSRP < buyPrice THEN
           BEGIN
           SELECT CONCAT('Check constraint on products.buyPrice &
products.MSRP failed – MSRP ',MSRP,' must be larger than buyPrice
',buyPrice)
           INTO _messageText;
        SIGNAL SQLSTATE '45002'
            SET MESSAGE_TEXT = _messageText;
           END;
    END IF;

END$$
DELIMITER ;
```

A quick explanation of our stored procedure:
As with our stored functions and triggers we temporarily replace our delimiter.

```
DELIMITER $$
```

Our function is called `check_products` and takes in 2 parameters `buyPrice` and `MSRP` which are both `Decimal(10,2)`.

```
CREATE PROCEDURE `check_products`(IN buyPrice DECIMAL(10,2), IN MSRP
DECIMAL(10,2))
```

We declare a `_messageText` variable which will store an error message.

```
DECLARE _messageText varchar(250);
```

This part:

```
IF buyPrice < 0 THEN
           BEGIN
           SELECT CONCAT('Check constraint on products.buyPrice failed
- buyPrice ',buyPrice,' must be positive.')
           INTO _messageText;
        SIGNAL SQLSTATE '45001'
            SET MESSAGE_TEXT = _messageText;
           END;
    END IF;
```

Checks to see if the buyPrice is negative and if it is, Signal an SQLState of 45001 which will abort the current query and report back the error we've created - `Check constraint on products.buyPrice failed - buyPrice must be positive.`

We check once again in the same fashion that `MSRP < buyPrice` and abort (with Signal an SQLState of 45002) if MSRP is not larger than buyPrice with this message - `Check constraint on products.buyPrice & products.MSRP failed - MSRP  must be larger than buyPrice.`

Step 3:
We will create triggers now that call the `check_products` procedure whenever data is changed in the products table.

Create the BEFORE INSERT trigger:

```
DELIMITER $$
CREATE TRIGGER `products_before_insert` BEFORE INSERT ON `products`
FOR EACH ROW
BEGIN
    CALL check_products(new.buyPrice,new.MSRP);
END$$
DELIMITER ;
```

Create the BEFORE UPDATE trigger:

```
DELIMITER $$
CREATE TRIGGER `products_before_update` BEFORE UPDATE ON `products`
FOR EACH ROW
BEGIN
    CALL check_products(new.buyPrice,new.MSRP);
END$$
DELIMITER ;
```

Now let's test it:
First we'll try to insert an invalid row:

```
INSERT INTO products
(productCode, productName, productLine, productScale, productVendor,
productDescription, quantityInStock, buyPrice, MSRP)
VALUES
(GetProductId(), 'Stealth Bomber 2', 'Planes', '1:200', 'Army
Replicas', 'Northrop Grumman B-2 Spirit', 100, 52.50, 8.25);
```

Note that MSRP is too low.

We should get our error message back:
Submit your screenshot:
09_invalid_MSRP_insert.jpg

IMPORTANT! Ensure your screenshot shows the complete message!

We need to test the update trigger and check the buyPrice too low:

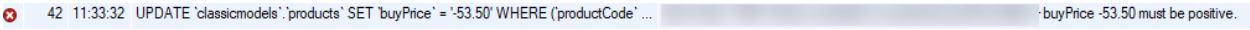
```
UPDATE `classicmodels`.`products` SET `buyPrice` = '-53.50' WHERE
(`productCode` = 'S23_5003');
```

NOTE: Since your `productCode` may be different for the `'Stealth Bomber'` row you just added, make sure you have the correct `productCode`. In my example it's **`'S23_5003'`** but for you it might be `'S`***`19`***`_500`***`3`***`'` or `'S`***`19`***`_500`***`6`***`'` or… (make sure it contains the correct year code `YY` and the current value of the incrementing number `500X`).

It should fail <u>and</u> you should see your custom error message - 'buyPrice must be positive':
Submit your screenshot:

10_invalid_buyPrice_update.jpg

Submit



IMPORTANT! Ensure your screenshot shows the complete message!